



REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
ASIGNATURA: INGENIERÍA DE SOFTWARE II

***GESTIÓN DE CONFIGURACIÓN DE SOFTWARE (SCM), CONTROL DE
VERSIONES AVANZADO Y PARADIGMAS DEVOPS EN ENTORNOS DE ALTO
RENDIMIENTO***

Profesor:

Mg. Félix Márquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

Ciudad Guayana, septiembre de 2026

Introducción

La Gestión de Configuración de Software (Software Configuration Management, SCM) ha experimentado una evolución paradigmática sustancial a lo largo de las últimas dos décadas. Históricamente, marcos normativos clásicos como el estándar IEEE SWEBOOK formalizaron la SCM como una disciplina administrativa centrada en el control formal de líneas base estáticas, inventarios de ítems de configuración (CI) y la fiscalización documental a cargo de comités burocráticos conocidos como Juntas de Control de Cambios (Change Control Boards, CCB).

En el contexto contemporáneo de la ingeniería de software, dominado por la computación en la nube, las arquitecturas distribuidas de microservicios y la necesidad de feedback continuo, el concepto de configuración ha trascendido la administración aislada del código fuente. La consolidación de la filosofía Everything as Code (EaC) postula que la infraestructura física y lógica, las políticas de seguridad corporativas, los entornos operativos de ejecución y las propias tuberías de entrega continua deben ser formalizadas mediante especificaciones declarativas legibles por máquina, versionadas y gobernadas con rigurosa inmutabilidad criptográfica (Humble & Farley, 2010).

El presente informe técnico desarrolla de manera exhaustiva y sin omisiones la resolución formal de la evaluación correspondiente a la Unidad V de Ingeniería de Software II. A lo largo del documento se abordan de manera crítica los pilares fundamentales del área: la orquestación concurrente de flujos de trabajo en Git bajo el modelo GitHub Flow frente a fallos bloqueantes en producción; el diseño estructurado de plantillas para Pull Requests y la formulación de compuertas de calidad (Quality Gates) en pipelines automatizados de CI/CD; la erradicación definitiva de la deriva de configuración (configuration drift) mediante la contenedorización con Docker; la comparativa arquitectónica entre GitFlow y Trunk-Based Development a la luz del análisis empírico del modelo DORA y el libro Accelerate (Forsgren et al., 2018).

Desarrollo de la Evaluación

Pregunta 1. Flujo de Trabajo en Git y Pull Requests (Control de Cambios Moderno)

Contexto Operativo: Se asume el rol de liderazgo técnico dentro de una organización que desarrolla y opera un sistema SaaS de facturación empresarial bajo el modelo de ramificación GitHub Flow (rama principal main protegida y ramas de corta duración para funcionalidades). De forma concurrente, un ingeniero de desarrollo ha concluido la implementación de la nueva funcionalidad de "exportación a PDF" en su rama local feature/pdf-export. De manera simultánea, el sistema de observabilidad y telemetría en producción notifica una falla crítica de prioridad máxima (P0/Sev-1): un defecto grave introducido en la lógica de autenticación en la rama main bloquea totalmente el inicio de sesión a todos los usuarios del sistema.

Secuencia Rigurosa de Comandos Git y Acciones en la Plataforma

El manejo de esta situación demanda una separación absoluta de ambos flujos de trabajo para evitar la fuga de código no validado a producción y reducir al mínimo el Tiempo Medio de Reparación (MTTR) del servicio:

a) Publicación segura de la funcionalidad (feature/pdf-export):

El desarrollador de la funcionalidad no debe intentar integrar su código de manera precipitada en una rama troncal que se encuentra fallando en producción. Debe asegurar sus cambios localmente, sincronizar su espacio de trabajo y publicar su rama al repositorio remoto para iniciar la revisión sin alterar main:

```
# 1. Verificar el estado del arbol de trabajo local y confirmar los cambios
git status
git add .
git commit -m "feat(invoice): implement core pdf export service and streaming response"

# 2. Descargar las ultimas referencias remotas sin fusionar
git fetch origin
```

```
# 3. Rebasar la rama sobre el commit base estable de origin/main
git rebase origin/main
```

```
# 4. Publicar la rama de funcionalidad en el repositorio remoto
git push -u origin feature/pdf-export
```

Posteriormente, el desarrollador procede en la interfaz de GitHub/GitLab a aperturar el Pull Request apuntando desde feature/pdf-export hacia main. En el título y descripción se debe agregar la etiqueta informativa *[WIP / Blocked by Hotfix]* para congelar su fusión hasta que la estabilidad de producción sea restaurada formalmente. *(Nota: El diagrama de flujo cronológico y topológico que detalla visualmente la bifurcación, estabilización de main y el aislamiento de ramas ha sido desarrollado e incorporado en los anexos gráficos del equipo).*

b) Aplicación, verificación y despliegue del Hotfix de producción:

La restitución de la autenticación de usuarios representa la prioridad operativa absoluta. La corrección debe originarse directamente a partir de la versión exacta que corre en producción para evitar introducir cambios no testeados:

```
# 1. Cambiar a la rama principal y asegurar sincronización exacta con producción
git checkout main
git pull origin main
```

```
# 2. Crear una rama de remediación urgente directamente desde main
git checkout -b hotfix/auth-login-bypass
```

```
# 3. Realizar la corrección mínima indispensable y agregar pruebas de regresión
# (Se modifica el controlador de autenticación y su archivo de pruebas unitarias)
git add src/auth/auth.controller.ts src/auth/auth.controller.spec.ts
git commit -m "fix(auth): correct token validation claims preventing user login"
```

```
# 4. Publicar la rama de hotfix en el repositorio remoto
git push -u origin hotfix/auth-login-bypass
```

En la plataforma de gestión se crea de inmediato un Pull Request prioritario desde hotfix/auth-login-bypass hacia main. Se disparan automáticamente los pipelines de CI con prioridad de cola. Una vez que se obtienen las validaciones exitosas de los Quality Gates y la

aprobación rápida de dos revisores técnicos (peer reviews), se procede a ejecutar el merge directo (Squash and Merge o Rebase and Merge). El pipeline de Entrega/Despliegue Continuo (CD) libera automáticamente el parche a producción, y se genera la etiqueta criptográfica correspondiente:

```
git checkout main
git pull origin main
git tag -a v2.4.1 -m "hotfix: resolve critical authentication login blockage"
git push origin v2.4.1
```

Una vez resuelta la contingencia en producción, el desarrollador que esperaba con la funcionalidad de PDF procede a incorporar la corrección en su espacio de trabajo ejecutando: `git checkout feature/pdf-export && git fetch origin && git rebase origin/main`. De este modo, la rama `feature/pdf-export` queda al día con el parche de seguridad sin generar divergencias históricas ni conflictos de integración.

Diseño de Plantilla Formal de Pull Request (Template de PR)

Para estandarizar el proceso de revisión por pares y garantizar la gobernanza del repositorio, se establece la siguiente plantilla obligatoria en formato Markdown dentro del directorio `.github/pull_request_template.md`:

```
## 1. Descripcion del Cambio
- **Modulo Afectado:** Modulo de Facturacion / Generacion de Reportes y Documentos.
- **Tipo de Modificacion:** [X] Nueva Funcionalidad (Feature) [ ] Refactorizacion
[ ] Correccion (Hotfix)
- **Resumen Tecnico:** Implementacion del servicio asincrono para la conversion de facturas
a formato PDF. Se integra el motor de renderizado basado en Chromium headless, la creacion
de plantillas Handlebars y la persistencia temporal en almacenamiento de objetos (Blob Storage).
- **Issue Relacionado:** Resuelve #INV-4029.

## 2. Alcance e Impacto Operativo
- **Impacto en Arquitectura:** La generacion se delega a colas de mensajeria (RabbitMQ/Redis)
para mitigar el consumo de CPU y memoria en el hilo principal del servidor HTTP.
```

- ****Impacto en Base de Datos:**** ☐ Requiere migraciones ☒ Sin modificaciones de esquema.

- ****Seguridad y Variables de Entorno:**** Requiere configurar la variable ``PDF_STORAGE_ENCRYPTION_KEY`` con cifrado AES-256 para la proteccion de facturas en reposo.

3. Checklist Estricto de Revision Tecnica y Calidad

- ☐ Cobertura de Pruebas: Se agregaron pruebas unitarias y de integracion que garantizan un

minimo del 85% de cobertura sobre el nuevo controlador de exportacion.

- ☐ Seguridad en Dependencias y Procesamiento de PDF:

- ¿Se audito rigurosamente la libreria generadora de PDF contra vulnerabilidades

conocidas de Inyeccion de HTML y Server-Side Request Forgery (SSRF)? ☒ Si.

- ¿El motor de renderizado implementa escape estricto de entidades HTML previo a la evaluacion? ☒ Si.

- ¿Se configuraron limites estrictos de timeout (max 8s) y consumo maximo de memoria RAM

por worker (cgroups) para neutralizar ataques de Denegacion de Servicio (DoS)? ☒ Si.

- ☐ Rendimiento y Fugas de Memoria: Se confirmo el cierre explicito de los contextos de navegacion

y buffers binarios tras cada descarga (``browser.close()``), eliminando posibles memory leaks.

- ☐ Conformidad de API: El endpoint implementado satisface las especificaciones contractuales OpenAPI v3.

4. Evidencia de Pruebas Locales

- Registro de ejecucion de pruebas automatizadas o enlace al entorno de validacion interna:

``http://qa-env.internal/reports/invoice_sample_4029.pdf``

Automatización de la Calidad (Quality Gates del Pipeline de CI)

En cumplimiento de las buenas prácticas de Integración Continua, el botón de Merge permanece condicionalmente deshabilitado en el repositorio por medio de políticas de ramas protegidas. El pipeline de CI debe verificar y aprobar de manera obligatoria las siguientes tres compuertas de calidad:

1. Compilación Exitosa y Cobertura Automatizada de Pruebas (Test & Coverage

Gate): El código debe compilar de forma limpia sin emitir advertencias de tipado o sintaxis (tratar warnings como errores). La totalidad de la suite de pruebas unitarias, de integración y de contrato debe ejecutarse en agentes limpios con un resultado de 100% de éxito. Además, la herramienta de análisis de cobertura (como Istanbul/Jest o JaCoCo) debe certificar que el código nuevo o modificado alcanza una cobertura de líneas de al menos un 80%, impidiendo que la cobertura global del proyecto disminuya respecto a la línea base previa.

2. Análisis Estático de Seguridad (SAST) y Composición de Software (SCA Security

Gate): El código fuente debe ser escaneado mediante herramientas de SAST (como SonarQube, Semgrep o Snyk Code) para validar la inexistencia de vulnerabilidades críticas pertenecientes al OWASP Top 10 (tales como inyecciones, referencias directas inseguras o ejecución remota de código). En simultáneo, un motor de SCA (Trivy, Dependabot o npm audit) debe analizar el árbol de dependencias directas y transitivas; se establece una regla de bloqueo que aborta el pipeline si se detecta cualquier vulnerabilidad conocida catalogada con nivel de severidad "High" o "Critical" (puntuación CVSS mayor o igual a 7.0).

3. Deuda Técnica y Estándares de Calidad de Código (Quality & Maintainability

Gate): El código debe cumplir estrictamente con los lineamientos de linters y analizadores estáticos. Se exige una tasa de duplicación de código del 0.0% en los módulos modificados, un índice de complejidad ciclomática de McCabe inferior a 10 en cada método y la obtención de una calificación global 'A' en el índice de mantenibilidad técnica, garantizando que no se introduzcan pasivos técnicos al repositorio.

Ejercicio 2. Everything as Code y Gestión de Entornos

Contexto Operativo: La organización enfrenta constantes fallos operativos atribuidos al síndrome de "funciona en mi máquina". Los desarrolladores estructuran sus entornos locales de manera heterogénea y discrecional, mientras que el personal de operaciones configura los servidores de producción mediante procedimientos manuales y scripts de Bash no versionados,

originando severas desviaciones y caídas del sistema.

Dockerización y Erradicación de la Deriva de Configuración

Con el objetivo de garantizar paridad absoluta y determinismo en todas las fases del ciclo de vida, se diseña el siguiente archivo Dockerfile optimizado en múltiples etapas (Multi-stage build) para una aplicación basada en Node.js:

```
# =====
# Dockerfile: Aplicacion Node.js (Servicio de Facturacion)
# Patron: Multi-stage build para seguridad y optimizacion de tamano
# =====

# -----
# ETAPA 1: Compilacion y preparacion de dependencias (Build Stage)
# -----
FROM node:20-alpine AS build

# Establecer directorio de trabajo en un espacio aislado
WORKDIR /usr/src/app

# Copiar manifiestos de dependencias para aprovechar la cache de capas de Docker
COPY package.json package-lock.json ./

# Instalacion determinista y limpia del arbol completo de dependencias
# 'npm ci' asegura que solo se instalen las versiones exactas del lockfile
RUN npm ci

# Copiar el resto del codigo fuente de la aplicacion
COPY . .

# Si la aplicacion requiere transpilacion de codigo (TypeScript), se ejecuta aqui:
# RUN npm run build

# Purgar las dependencias de desarrollo para aligerar la imagen de produccion
RUN npm prune --production

# -----
# ETAPA 2: Entorno minimo y seguro de ejecucion en produccion (Runtime Stage)
# -----
FROM node:20-alpine AS production

# Declarar la variable de entorno para optimizar rendimiento de Node.js
```



```
ENV NODE_ENV=production

# Establecer directorio de trabajo final
WORKDIR /usr/src/app

# Practica de Seguridad: Ejecutar bajo un usuario sin privilegios administrativos
# Se utiliza el usuario preexistente 'node' de la imagen Alpine en lugar de root
USER node

# Transferir desde la etapa de compilacion unicamente los artefactos requeridos
# Asignar la propiedad de los archivos directamente al usuario sin privilegios
COPY --chown=node:node --from=build /usr/src/app/node_modules ./node_modules
COPY --chown=node:node --from=build /usr/src/app/package.json ./package.json
COPY --chown=node:node --from=build /usr/src/app/server.js ./server.js

# Documentar explicitamente el puerto de red en el que escucha el servicio
EXPOSE 8080

# Comando inmutable para el arranque del contenedor
CMD ["node", "server.js"]
```

Explicación Técnica de la Erradicación de la Deriva de Configuración: La deriva de configuración (configuration drift) representa la divergencia inadvertida e incontrolada que sufren los entornos operativos a lo largo del tiempo debido a parches manuales, librerías del sistema dispares o diferencias en las variables ambientales del servidor anfitrión. El archivo Dockerfile desarrollado erradica por completo esta problemática al transformar todo el entorno de ejecución en un artefacto inmutable y autocontenido.

Al empaquetar en una sola imagen la versión inalterable del motor de ejecución (Node.js 20), las bibliotecas de enlace del sistema operativo base (Alpine Linux), la estructura determinista de módulos fijada en package-lock.json y el código fuente compilado, se elimina la dependencia en las particularidades del host. La imagen resultante cuenta con un identificador criptográfico único (digest SHA-256). El mismo artefacto que se valida en el entorno local del desarrollador y en el pipeline de CI es el que se traslada sin alteración alguna a producción, garantizando que el software funcione con idéntico comportamiento en cualquier infraestructura física o en la nube.

Reflexión: Trazabilidad y Auditoría en Everything as Code frente a Bitácoras Tradicionales

Bajo la óptica de los principios fundamentales de la SCM clásica (identificación formal, control de cambios, auditoría de configuración y estado de configuración), el tratamiento de la infraestructura como código (IaC) dentro de Git supera ampliamente a los modelos clásicos basados en bitácoras manuales de administradores de sistemas en tres dimensiones determinantes:

1. Trazabilidad Criptográfica Inmutable: En la administración tradicional, las bitácoras o logs de servidores describen eventos de forma posterior (post-mortem), discontinua y son vulnerables a manipulación intencionada o pérdidas accidentales. Por el contrario, en Git cada cambio sobre un contenedor, red o servidor se materializa en un commit atómico vinculado a un hash criptográfico SHA-1/SHA-256 inalterable, con firma digital GPG, fecha y hora exacta, e identidad certificada del autor, asociando la causa técnica del cambio a un Pull Request formalmente aprobado.

2. Auditoría Preventiva y Determinista: En lugar de efectuar auditorías forenses sobre servidores activos para descubrir qué configuraciones cambiaron, el paradigma Everything as Code permite auditar la infraestructura antes de que esta sea desplegada. Herramientas automatizadas de SAST para IaC inspeccionan los manifiestos en el pipeline de CI, comprobando el cumplimiento de directrices de seguridad y gobernanza corporativa sin depender de revisiones manuales tardías.

3. Resiliencia y Capacidad Inmediata de Reversión (Rollback): Recuperar un servidor alterado manualmente tras una falla operativa suele requerir horas de reconstrucción empírica de pasos previos no documentados. Con IaC y GitOps, el estado deseado del sistema se encuentra completamente definido en el repositorio. Ante cualquier anomalía, la restauración a un estado funcional previo se consigue de forma inmediata ejecutando un git revert, forzando a los agentes de reconciliación a restituir la configuración exacta en cuestión de minutos.

Pregunta 2. Análisis Crítico y Comparativo de Modelos de Ramificación (GitFlow vs. Trunk-Based Development)

La selección de la topología de ramificación (branching strategy) condiciona profundamente la dinámica de trabajo de los equipos, su velocidad de entrega y la calidad intrínseca del software desarrollado.

El Modelo Tradicional (GitFlow): Diseñado originalmente por Vincent Driessen en 2010, GitFlow establece un modelo estructurado y jerárquico apoyado en dos ramas permanentes de larga vida: main (reservada exclusivamente para versiones estables en producción) y develop (que aglutina los avances en integración continua). Este núcleo se complementa con ramas auxiliares temporales: ramas de funcionalidad (feature/*), ramas de estabilización de versión (release/*) y ramas de atención de emergencias (hotfix/*).

Si bien este modelo brindó certidumbre a organizaciones que producían software distribuido en medios físicos con calendarios de liberación prolongados (trimestrales o anuales), introduce severas ineficiencias en el contexto de la entrega continua. Fomenta el aislamiento prolongado de desarrolladores en ramas que duran semanas o meses, lo que acumula un elevado volumen de Trabajo en Proceso (WIP) no integrado. Al confluir estas ramas extensas hacia develop, los equipos caen inevitablemente en el denominado "infierno de integración" (merge hell): sesiones desgastantes de resolución de conflictos manuales, aparición de fallas de regresión complejas y retrasos sistemáticos en los plazos de entrega.

El Paradigma Contemporáneo: Trunk-Based Development (TBD): En contraste, el Desarrollo Basado en el Tronco plantea que los ingenieros integren sus cambios de manera directa sobre una única rama principal compartida (denominada trunk o main) múltiples veces al día, o mediante ramas de características sumamente efímeras cuya vida útil no excede de 24 a 48 horas. El fundamento central de TBD es que al integrar incrementos pequeños y frecuentes (commits atómicos), los conflictos de fusión se vuelven triviales y se detectan de manera casi instantánea.

Argumentación Basada en la Evidencia Científica de 'Accelerate': En el estudio científico sobre el desempeño de organizaciones de ingeniería de software expuesto por Nicole Forsgren, Jez Humble y Gene Kim en su libro *Accelerate* (2018), respaldado por los relevamientos anuales de las métricas DORA (DevOps Research and Assessment), se demuestra

de forma cuantitativa que la adopción de Trunk-Based Development es uno de los principales predictores estadísticos del alto rendimiento organizacional. Las organizaciones ubicadas en los percentiles de élite rechazan las ramas de larga duración por su impacto negativo comprobado en los cuatro indicadores clave de rendimiento:

1. *Frecuencia de Despliegue (Deployment Frequency)*: TBD habilita a las organizaciones a desplegar incrementos a producción múltiples veces al día, mientras que las estructuras burocráticas de GitFlow las restringen a ventanas semanales o mensuales.

2. *Tiempo de Espera para Cambios (Lead Time for Changes)*: Al reducir drásticamente la duración de las ramas, el intervalo transcurrido desde el primer commit hasta su puesta en funcionamiento productivo se reduce de varias semanas a pocas horas.

3. *Tasa de Falla en Cambios (Change Failure Rate)*: Modificaciones atómicas y reducidas presentan una superficie de fallo mucho menor, reduciendo la probabilidad de introducir defectos imprevistos en producción.

4. *Tiempo Medio de Recuperación (Time to Restore Service - MTTR)*: Diagnosticar y remediar una falla sobre un commit pequeño es significativamente más sencillo y rápido que realizar ingeniería inversa sobre paquetes masivos de lanzamiento consolidados en GitFlow.

Prácticas Obligatorias de SCM para la Viabilidad de Trunk-Based Development: Para que la integración directa sobre el tronco no comprometa la estabilidad del código en producción, resulta imprescindible la coexistencia de tres prácticas técnicas rigurosas:

a) *Batería Exhaustiva de Pruebas y Pipeline de CI Automatizado*: Cada commit sobre el tronco debe disparar un pipeline ultraveloz (que concluya en menos de 10 minutos) que ejecute miles de pruebas unitarias, de integración y compuertas de calidad. Si el tronco se rompe, la política de ingeniería exige que la reparación inmediata o la reversión del commit sea la prioridad número uno del equipo.

b) *Banderas de Características (Feature Flags / Feature Toggles)*: Constituyen el mecanismo neurálgico que permite desacoplar el *despliegue* técnico de código de la *liberación*

efectiva de funcionalidades para los usuarios de negocio. Los ingenieros pueden enviar código incompleto al tronco y desplegarlo en producción manteniendo la funcionalidad inactiva mediante condicionales lógicos en tiempo de ejecución, facilitando además despliegues canario (canary releases) y pruebas A/B de manera segura.

c) Revisiones de Código Ágiles y Continuas: Al trabajar con ramas de vida muy corta y pull requests atómicos, las revisiones por pares dejan de ser inspecciones prolongadas y tediosas para transformarse en validaciones colaborativas asincrónicas resueltas en minutos.

Pregunta 3. Evolución del Control de Cambios: La CCB Tradicional frente al Pipeline Automatizado

Discusión Crítica: La afirmación que sostiene que *"La Junta de Control de Cambios (CCB/CAB) tradicional es un cuello de botella que viola los principios de la Entrega Continua"* es plenamente acertada desde el punto de vista de la ingeniería moderna de software. Las CCB nacieron bajo esquemas predictivos con la premisa de mitigar riesgos a través de la revisión manual y la deliberación de comités de gestión que se reunían de forma periódica (semanal o quincenalmente) para autorizar despliegues.

Sin embargo, la experiencia ha evidenciado que la inspección humana sobre formularios descriptivos o actas de cambio no posee capacidad real para detectar condiciones de carrera, errores de concurrencia o incompatibilidades complejas en arquitecturas de software modernas. Paradójicamente, la espera impuesta por la CCB incentiva a los equipos a agrupar grandes volúmenes de código en lanzamientos gigantescos (large batch sizes), lo que incrementa sustancialmente el riesgo técnico, multiplica los incidentes y encarece la resolución de problemas (Humble & Farley, 2010).

La Dupla Pull Request + Pipeline de CI como Control de Cambios Automatizado: En la cultura DevOps moderna, el control de cambios no se elimina ni se descuida; por el contrario, se fortalece al transformarse en un proceso algorítmico, determinista e inmutable que supera al proceso manual en tres aspectos esenciales:

1. Mayor Rigor Técnico y Objetividad: Mientras que un comité administrativo evalúa solicitudes de manera superficial, un pipeline de CI somete cada modificación a miles de verificaciones rigurosas: compilación estricta, cobertura de pruebas, escaneo de vulnerabilidades conocidas y cumplimiento de estándares de arquitectura. La evaluación es objetiva, reproducible e inmune al sesgo o agotamiento humano.

2. Reducción Radical del Tiempo de Retroalimentación: Frente a las semanas de latencia burocrática de la CCB, el pipeline de CI y las revisiones por pares en el Pull Request proveen retroalimentación al desarrollador en cuestión de minutos. Los defectos se descubren y corrigen en el instante preciso de su desarrollo, abaratando exponencialmente el costo de corrección según la curva clásica de Boehm (IEEE Computer Society, 2014).

3. Trazabilidad y Cumplimiento Normativo Superior: Cada integración autorizada en el repositorio cuenta con el registro inmutable de la firma del autor, los identificadores de commit en Git, las aprobaciones formales de pares en la plataforma y el log criptográfico de las compuertas superadas en el pipeline. Este registro ofrece una evidencia de auditoría verificable y transparente, idónea para satisfacer los requerimientos regulatorios más exigentes.

Pregunta 4. Artefactos Modernos y Auditoría en la Era DevOps

En los modelos clásicos de SCM, el "Plan de Gestión de Configuración de Software" solía consistir en un voluminoso documento de procesador de palabras que quedaba desactualizado al poco tiempo de redactarse. En los entornos modernos bajo la premisa de Everything as Code, este plan se materializa en archivos de texto plano alojados en el propio repositorio, versionados y ejecutados directamente por motores automatizados. A continuación se detallan cuatro artefactos esenciales:

1. Archivo .gitignore (Aislamiento y Definición de la Línea Base):

Función y Rol en SCM: Delimita con exactitud qué elementos del espacio de trabajo deben formar parte del inventario versionado y cuáles deben ser categóricamente ignorados por el sistema de control de versiones. Evita activamente que archivos resultantes de la compilación

(dist/, build/), volcados de depuración, registros de log y paquetes voluminosos de dependencias locales (como node_modules/ o entornos virtuales) contaminen el repositorio.

Garantía de Integridad y Control: Mantiene la ligereza del repositorio y evita inconsistencias entre sistemas operativos ocasionadas por binarios compilados localmente. Asimismo, desempeña un papel primordial en la seguridad de la SCM al impedir que archivos de variables de entorno con secretos sensibles (como .env) se incorporen por error al historial público o compartido del código.

2. Archivo Dockerfile (Especificación Declarativa del Entorno de Ejecución):

Función y Rol en SCM: Reemplaza los extensos manuales técnicos de aprovisionamiento de servidores por una receta declarativa y reproducible de las capas del sistema operativo, paquetes base, configuración de red, permisos de usuario y dependencias del sistema requeridas para ejecutar el software.

Garantía de Integridad y Control: Asegura la inmutabilidad y la repetibilidad del entorno de ejecución. Cualquier instancia creada a partir de este manifiesto genera un contenedor idéntico byte por byte, evitando la divergencia de librerías subyacentes del host. Permite auditar formalmente la composición del sistema mediante escáneres estáticos de contenedores antes de autorizar su paso a producción.

3. Archivo .github/workflows/ci.yml (Gobernanza Automatizada y Compuertas de Calidad):

Función y Rol en SCM: Este manifiesto estructurado en formato YAML codifica de manera formal el flujo de Integración y Despliegue Continuo. Define los eventos disparadores (triggers como push o pull_request), la matriz de agentes efímeros de compilación y la secuencia obligatoria de Quality Gates que deben superarse.

Garantía de Integridad y Control: Materializa de manera ejecutable las políticas de gobernanza que antes custodiaban los comités manuales de cambios. El archivo ci.yml impone barreras inviolables: ningún cambio de código puede integrarse a las ramas protegidas si el

pipeline falla. Además, garantiza trazabilidad al registrar qué versión exacta del pipeline validó cada uno de los commits en la historia del proyecto.

4. Archivo `package-lock.json` (o `Pipfile.lock` / `terraform.lock.hcl`):

Función y Rol en SCM: Los archivos de bloqueo (lockfiles) documentan el grafo exhaustivo y detallado de todas las dependencias directas y transitivas requeridas por el proyecto, fijando sus versiones exactas y sus sumas de verificación criptográfica (hashes de integridad SHA-512).

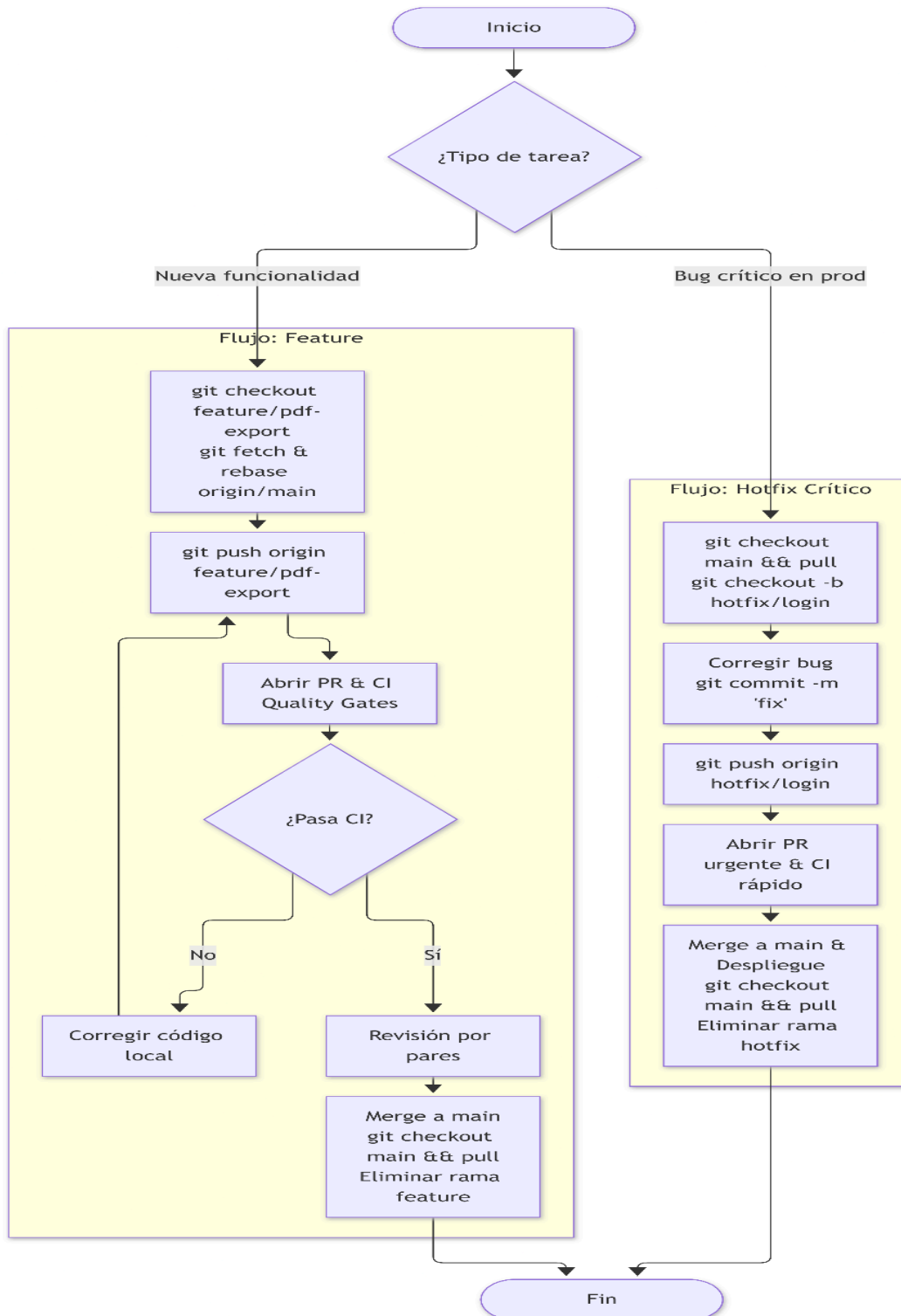
Garantía de Integridad y Control: Mientras que manifiestos abiertos como `package.json` manejan rangos semánticos flexibles (como `^2.1.0`), el lockfile congela la resolución de paquetes. Esto garantiza que dos compilaciones ejecutadas en diferentes equipos o momentos en el tiempo instalen exactamente los mismos paquetes binarios, blindando la cadena de suministro de software (Software Supply Chain) contra la inyección de código malicioso o fallos imprevistos derivados de actualizaciones no deseadas de terceros.

Conclusiones

La resolución exhaustiva de la evaluación de la Unidad V confirma que la Gestión de Configuración de Software no ha perdido vigencia en los entornos ágiles y DevOps, sino que ha evolucionado hacia un modelo mucho más formal, automatizado e integrado en la ingeniería del software moderno. La convergencia hacia flujos de trabajo eficientes como GitHub Flow y Trunk-Based Development, apoyada en la inmutabilidad que brindan los contenedores Docker y la validación determinista de pipelines de Integración Continua, permite que las organizaciones concilien la máxima velocidad de despliegue con elevados índices de estabilidad y resiliencia.

La sustitución de procedimientos burocráticos tradicionales por compuertas de calidad automáticas en los Pull Requests no debilita el control de cambios, sino que lo optimiza con criterios cuantitativos, inmediatos y auditables criptográficamente. Asimismo, la transición hacia el paradigma Everything as Code garantiza que cualquier componente de infraestructura, seguridad o entorno pueda ser reproducido y auditado con total fidelidad, sentando las bases operativas indispensables para el éxito de cualquier proyecto de software de alto impacto.

Anexos



Referencias

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Driessen, V. (2010). *A successful Git branching model*. Publicación técnica en línea. Recuperado de <https://nvie.com/posts/a-successful-git-branching-model/>
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and devops: Building and scaling high performing technology organizations*. IT Revolution Press.
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- IEEE Computer Society. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V3.0)*. IEEE Computer Society Press.
- Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Prokic, S. (2021). *Infrastructure as Code: Dynamic Infrastructure with Terraform*. Packt Publishing.